Confusion and Diffusion

Confusion

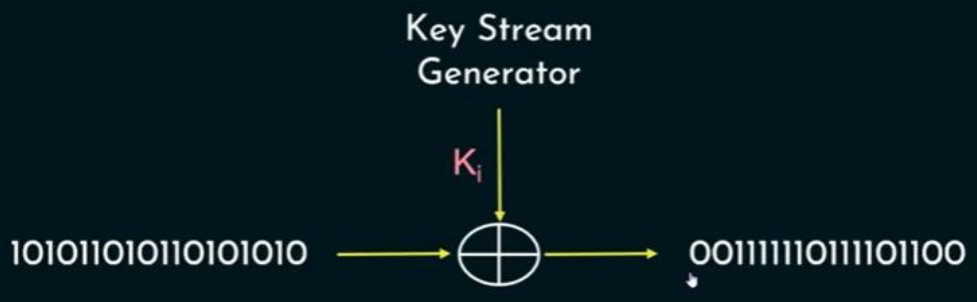
- ★ Making the relationship between the encryption key and the ciphertext as complex as possible.
- ★ Relationship between CT and PT is obscured.
- ★ Given CT, no info about PT, Key, Encryption algorithm etc.,
- ★ Example: Substitution.

Diffusion

- ★ Making each plaintext bit affect as many ciphertext bits as possible.
- ★ 1 bit change in PT, significant effect on CT.
- ★ Example: Transposition or Permutation.

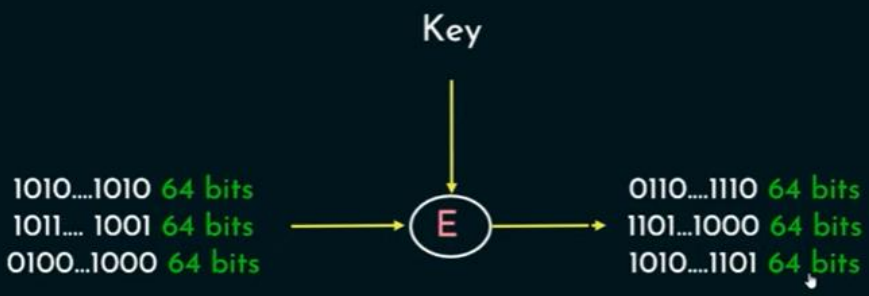
Stream Cipher

Each plaintext digit is encrypted one at a time with the corresponding digit of the keystream, to give a digit of the ciphertext stream.



Block Cipher

A block cipher is a deterministic algorithm operating on fixed-length groups of bits, called blocks.



Stream Cipher and Block Cipher

	Stream Cipher	Block Cipher
Length	Bit or Byte	Block Size – 64 bits, 128 bits
Design	Complex	Simple
Principle	Confusion	Confusion and Diffusion
Speed	Faster	Slower
Encryption	CFB (Cipher Feedback) and OFB (Output Feedback)	Electronic Code Book (ECB) and Cipher Block Chaining (CBC)
Decryption	XOR	Reverse of encryption
Example	Vernam Cipher	DES, AES

DIFFERENCE BETWEEN BLOCK AND STREAM CIPHER:

BLOCK CIPHER	STREAM CIPHER
1) it converts plain text to cipher by taking plain text block at a time.	1) it converts by taking 1 byte of plain text at a time
2) it is slow when compared to stream cipher.	2) it is fast when compared to block cipher.
3) uses both confusion and diffusion.	3) uses confusion but not diffusion.
4)in block cipher, reversibility of encrypted text is hard.	4) in stream cipher, reversibility of encrypted text is easy. [uses XOR for encryption]
5)64 or more than 64 bits are used.	5) 8 bits are used.
6) it is simple.	6) it is more complex.
7) One time pad is the best example of block cipher.	7) DES is the best example of stream cipher.

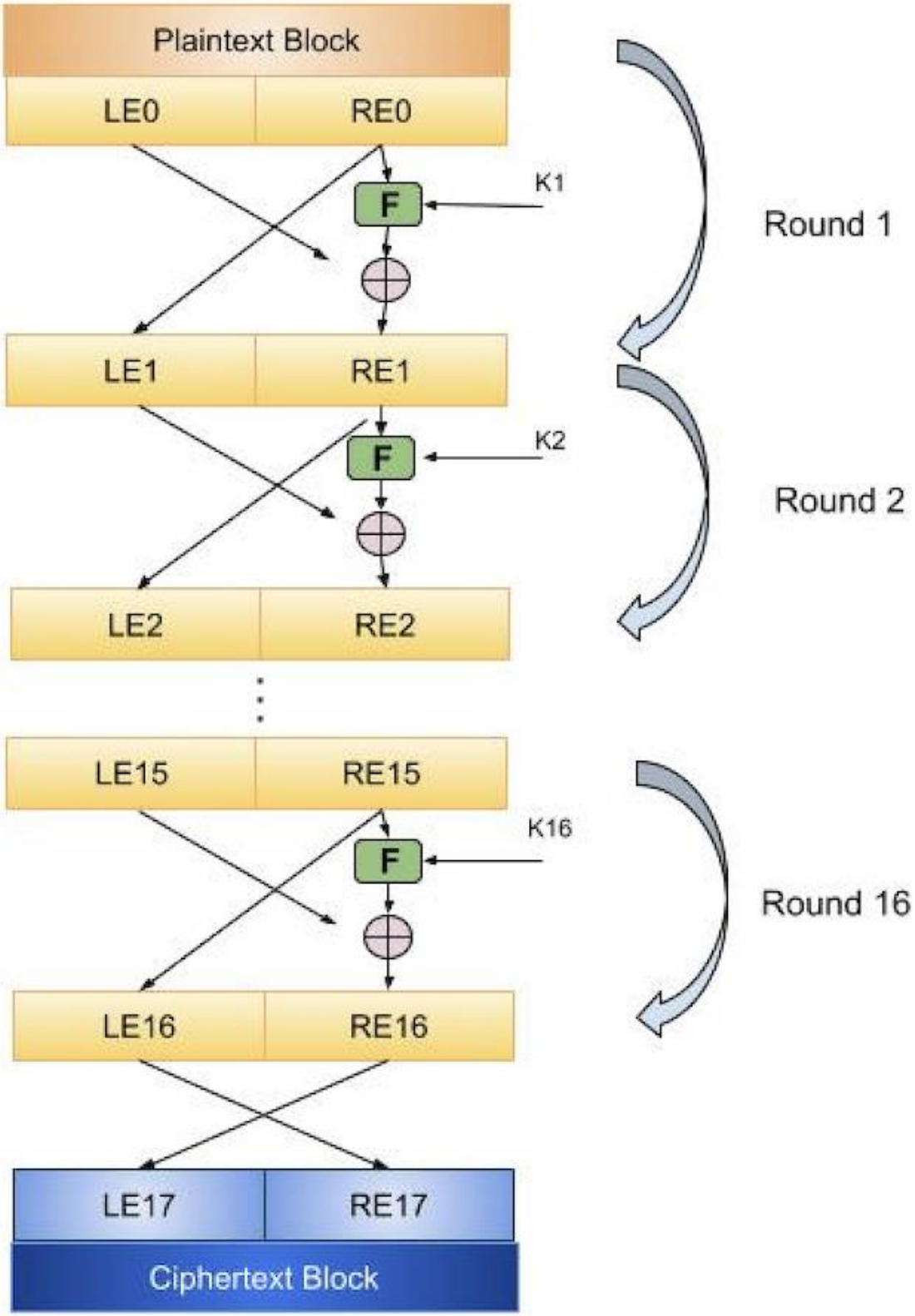
Feistel Block Cipher

Feistel Cipher is not a specific scheme of block cipher. It is a design model from which many different block ciphers are derived. DES is just one example of a Feistel Cipher. A cryptographic system based on Feistel cipher structure uses the same algorithm for both encryption and decryption.

Encryption Process

The encryption process uses the Feistel structure consisting multiple rounds of processing of the plaintext, each round consisting of a “substitution” step followed by a permutation step.

Feistel Structure is shown in the following illustration −


Plaintext Block
LE0
RE0
K1
F
Round 1
LE1
RE1
K2
F
Round 2
LE2
RE2
LE15
RE15
K16
F
Round 16
LE16
RE16
LE17
RE17
Ciphertext Block

- The input block to each round is divided into two halves that can be denoted as L and R for the left half and the right half.
- In each round, the right half of the block, R, goes through unchanged. But the left half, L, goes through an operation that depends on R and the encryption key. First, we apply an encrypting function 'f' that takes two input − the key K and R. The function produces the output f(R,K). Then, we XOR the output of the mathematical function with L.
- In real implementation of the Feistel Cipher, such as DES, instead of using the whole encryption key during each round, a round-dependent key (a subkey) is derived from the encryption key. This means that each round uses a different key, although all these subkeys are related to the original key.
- The permutation step at the end of each round swaps the modified L and unmodified R. Therefore, the L for the next round would be R of the current round. And R for the next round be the output L of the current round.
- Above substitution and permutation steps form a 'round'. The number of rounds are specified by the algorithm design.
- Once the last round is completed then the two sub blocks, 'R' and 'L' are concatenated in this order to form the ciphertext block.

The difficult part of designing a Feistel Cipher is selection of round function 'f'. In order to be unbreakable scheme, this function needs to have several important properties that are beyond the scope of our discussion.

Decryption Process

The process of decryption in Feistel cipher is almost similar. Instead of starting with a block of plaintext, the ciphertext block is fed into the start of the Feistel structure and then the process thereafter is exactly the same as described in the given illustration.

The process is said to be almost similar and not exactly same. In the case of decryption, the only difference is that the subkeys used in encryption are used in the reverse order.

The final swapping of 'L' and 'R' in last step of the Feistel Cipher is essential. If these are not swapped then the resulting ciphertext could not be decrypted using the same algorithm.

Design features

Feistel cipher design features that are considered when using block ciphers:

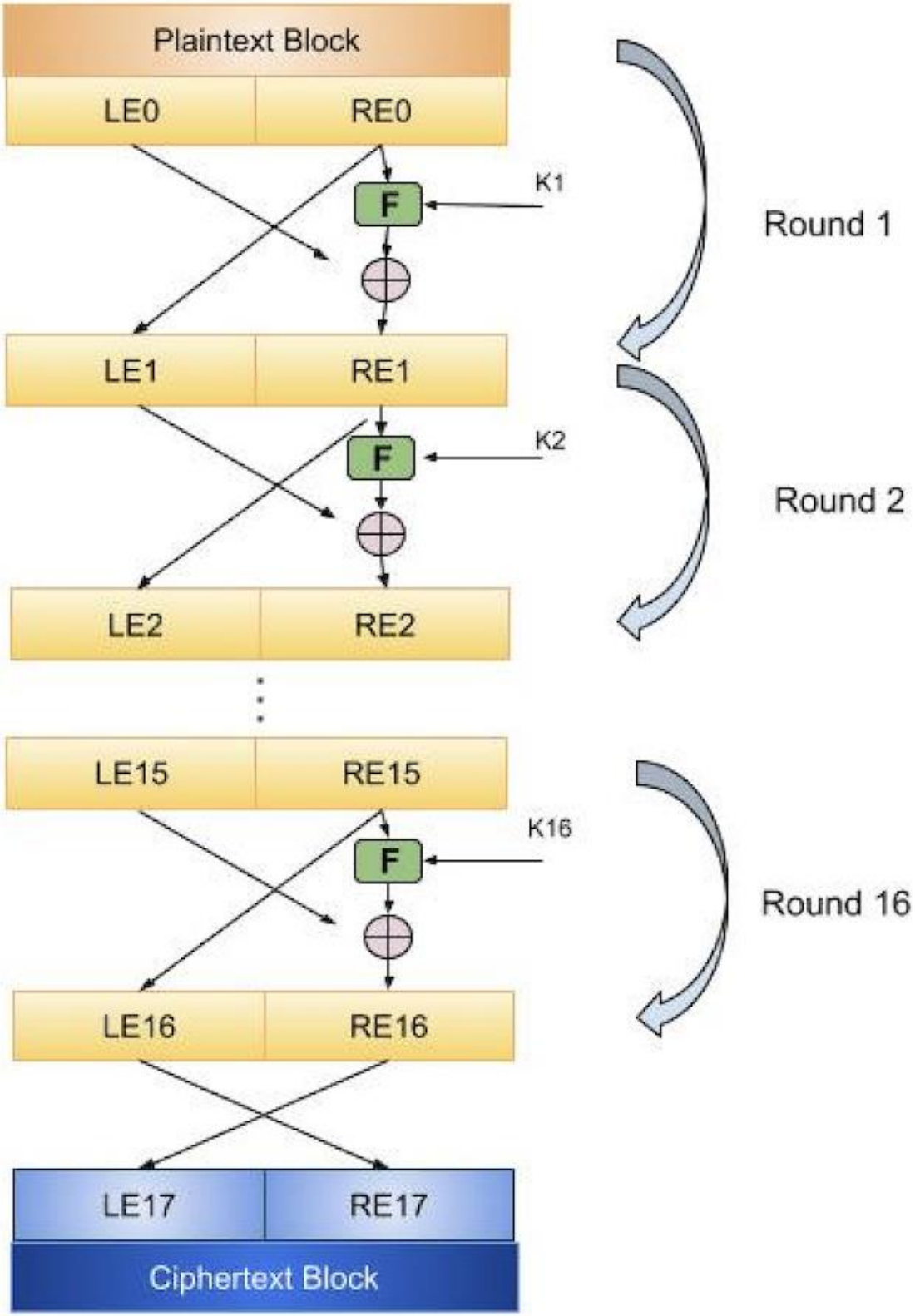
- **Block size –** Block ciphers are considered more secure when the block size is larger. Though, larger block sizes reduce the execution speed for the encryption and decryption process. Typically, block ciphers have a block size of 64-bits, but modern blocks like AES (Advanced Encryption Standard) are 128-bits.
- **Easy analysis –** Block ciphers should be easy to analyze, which can help identify and address any cryptanalytic weaknesses to create more robust algorithms.
- **Key size –** Like the block size, larger key sizes are considered more secure at the cost of potentially slowing down the time it takes to finish the encryption and decryption process. Modern ciphers use a 128-bit key, which has replaced the earlier 64-bit version.
- **The number of rounds –** The number of rounds can also impact the security of a block cipher. While more rounds increase security, the cipher is more complex to decrypt. Thus, the number of rounds depends on a business's desired level of data protection.
- **Round function –** A complex round function helps boost the block cipher's security.
- **Subkey generation function –** The more complex a subkey generation function is, the more difficult it is for expert cryptanalysts to decrypt the cipher.
- **Quick software encryption and decryption –** It's helpful to use a software application that can help produce faster execution speeds for block ciphers.

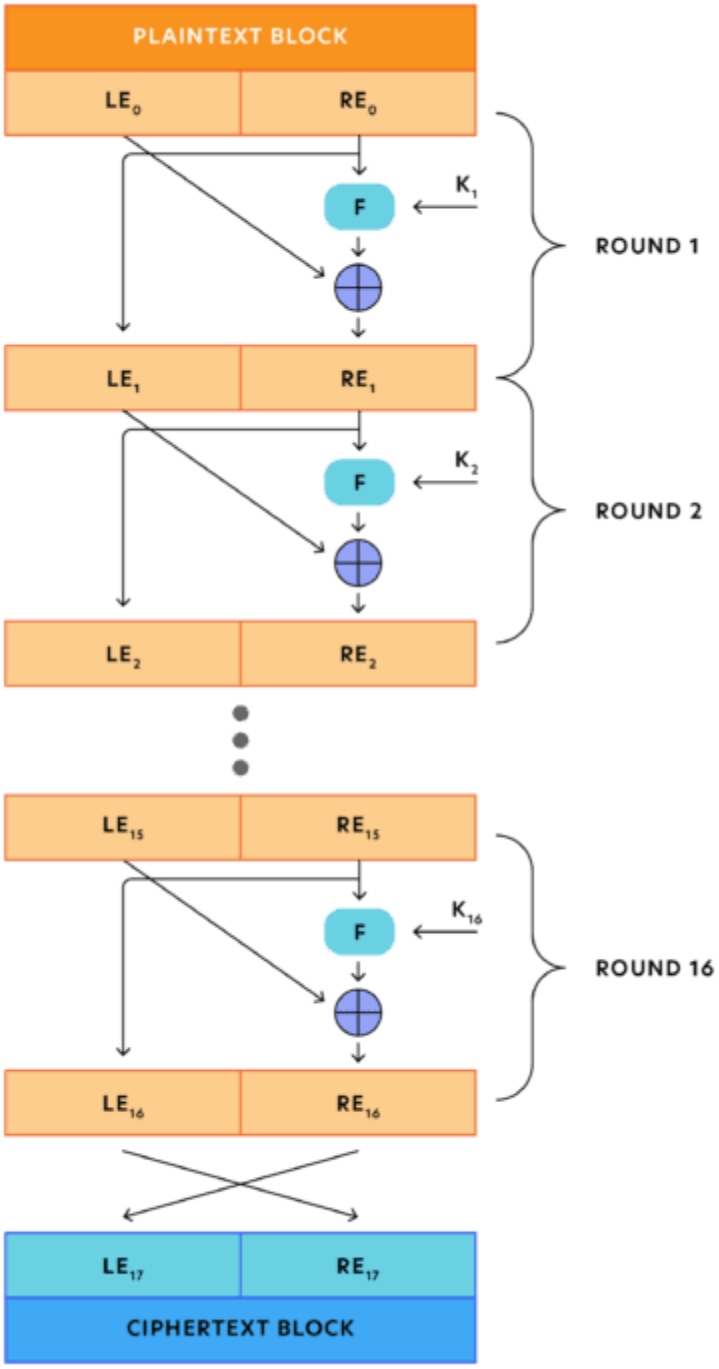
Design features

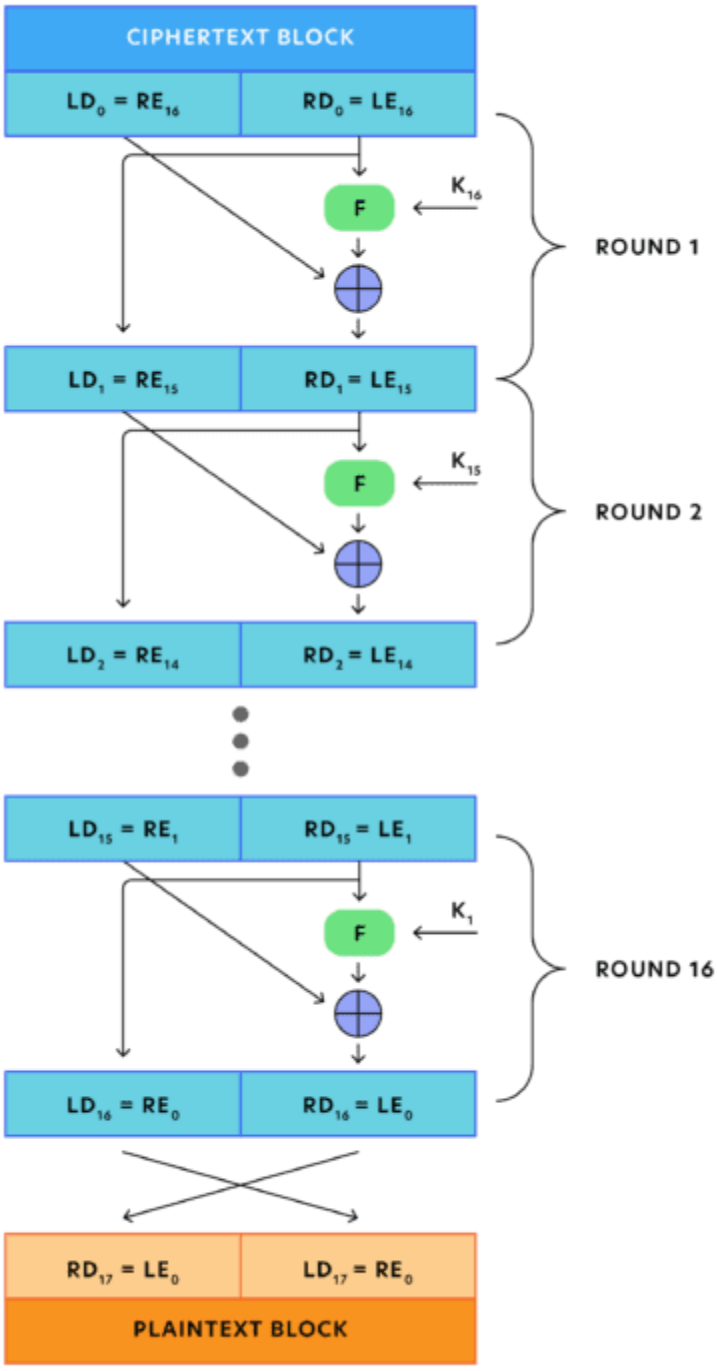
Feistel cipher structure has the following five components:

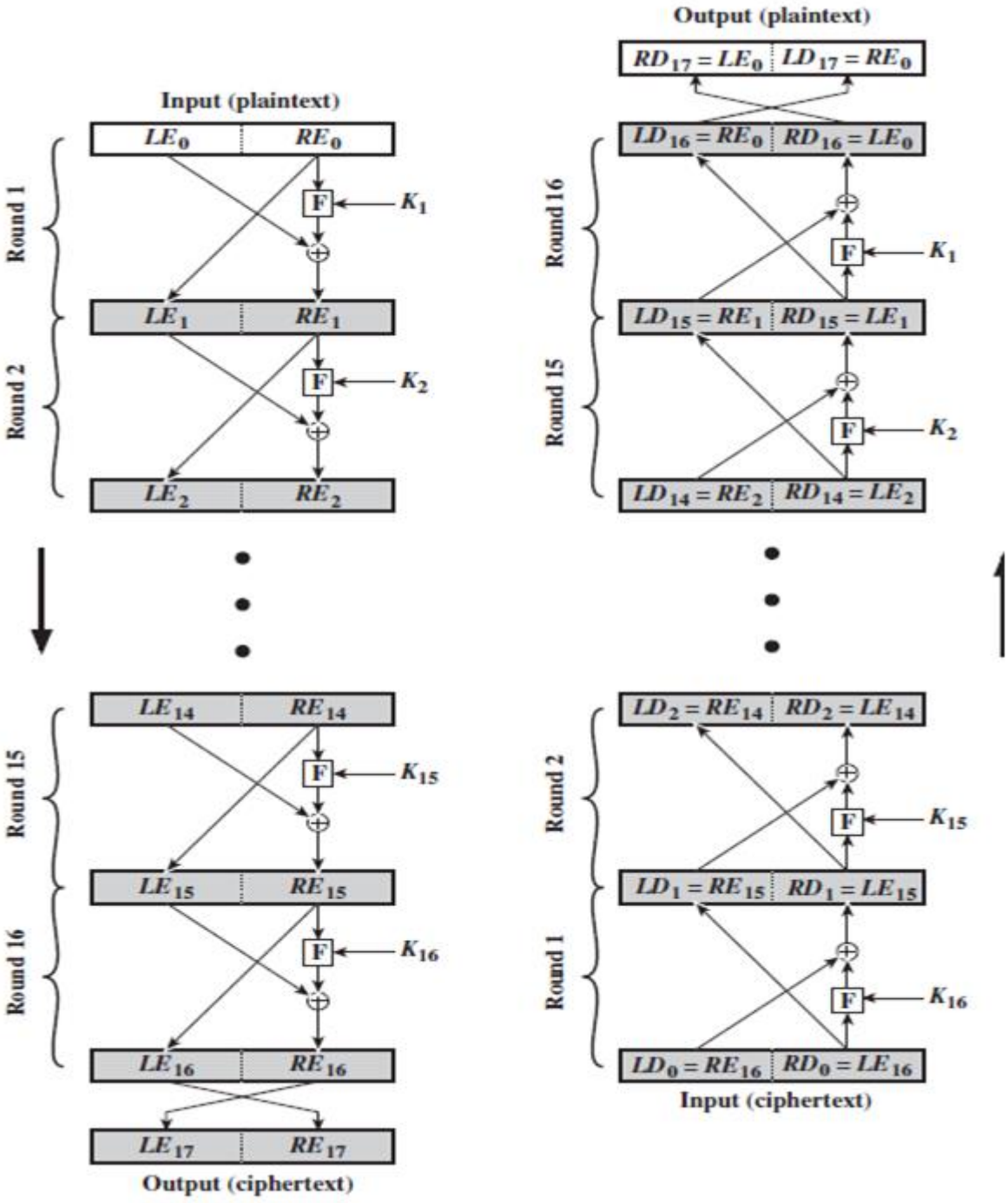
- **Number of rounds:** The greater the number of rounds, *n*, used for the encryption/decryption process, the higher the complexity; hence, the security of the block cipher.
- **Sub key generation algorithm:** Complex algorithms make it difficult for intruders to crack the key.
- **Encryption function:** Complex functions enhance the security of the block cipher, making them difficult to crack.

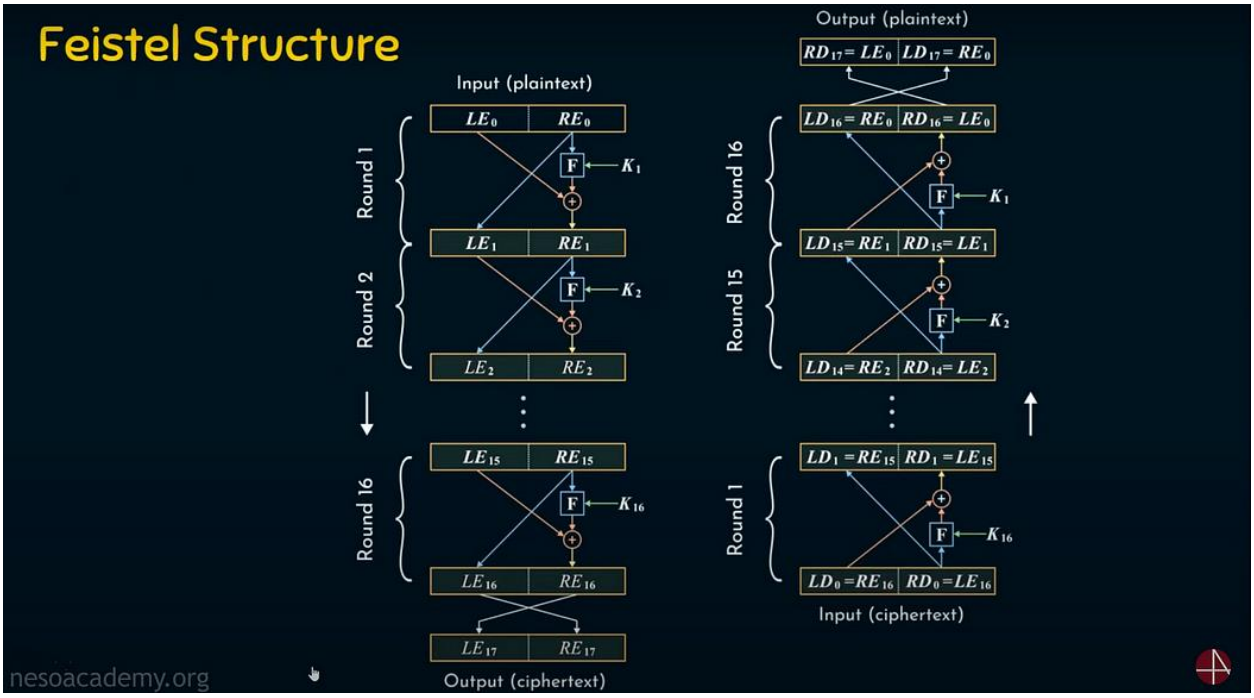
- **Block size:** The larger the size of the block, the more secure and complex the block cipher is. However, a larger block size reduces the execution speed of the encryption and decryption process.
- **Key size:** A large size key increases the security of the block cipher. However, it also makes the encryption and decryption process slow.


Plaintext Block
LE0
RE0
K1
F
Round 1
LE1
RE1
K2
F
Round 2
LE2
RE2
LE15
RE15
K16
F
Round 16
LE16
RE16
LE17
RE17
Ciphertext Block


PLAINTEXT BLOCK
LE0
RE0
K1
F
ROUND 1
LE1
RE1
K2
F
ROUND 2
LE2
RE2
LE15
RE15
K16
F
ROUND 16
LE16
RE16
LE17
RE17
CIPHERTEXT BLOCK


CIPHERTEXT BLOCK
LD0 = RE16
RD0 = LE16
F
K16
ROUND 1
LD1 = RE15
RD1 = LE15
F
K15
ROUND 2
LD2 = RE14
RD2 = LE14
LD15 = RE1
RD15 = LE1
F
K1
ROUND 16
LD16 = RE0
RD16 = LE0
RD17 = LE0
LD17 = RE0
PLAINTEXT BLOCK


Input (plaintext)
LE_0
RE_0
F
K_1
LE_1
RE_1
F
K_2
LE_2
RE_2
Round 1
Round 2
LE_{14}
RE_{14}
F
K_{15}
LE_{15}
RE_{15}
F
K_{16}
LE_{16}
RE_{16}
LE_{17}
RE_{17}
Output (ciphertext)
Round 15
Round 16
Output (plaintext)
$RD_{17} = LE_0$
$LD_{17} = RE_0$
$LD_{16} = RE_0$
$RD_{16} = LE_0$
F
K_1
$LD_{15} = RE_1$
$RD_{15} = LE_1$
F
K_2
$LD_{14} = RE_2$
$RD_{14} = LE_2$
$LD_2 = RE_{14}$
$RD_2 = LE_{14}$
F
K_{15}
$LD_1 = RE_{15}$
$RD_1 = LE_{15}$
F
K_{16}
$LD_0 = RE_{16}$
$RD_0 = LE_{16}$
Input (ciphertext)
Round 16
Round 15
Round 2
Round 1



Introduction

The Feistel structure's significance lies in its ability to enhance encryption security by incorporating confusion and diffusion techniques, which are crucial for obscuring data patterns and preventing unauthorized access. This article aims to provide an in-depth exploration of the Feistel cipher structure, drawing on a detailed tutorial from Neso Academy. By breaking down the encryption and decryption processes, examining the role of the 'F' function, and discussing design features such as key size and round operations.

Encryption and Round Operation:

This section describes the round operation that takes 64-bit input length plain text and divides it into two 32-bits and the input passes through the round operations (from round 1 to round 16) and finally the cipher text is produced.

3.1 Round 1 Operation:

1. Original 64-bit plain text is split into two 32-bit length text
2. The split two 32-bit length text are LE_o and RE_o.

3. The RE_o is directly given to the LE_1.
4. The REo is given to the function ‘F’ which performs some operation with the key K_1 and the output of the Function ‘F’ is again XORed with the LE_o and again its output is RE_1.
5. LE_1 is the 32-bit and RE_1 is the 32-bit that makes it 64-bit length.
6. In the Round 1 operation K_1 is used with the F so we call it round key for Round 1 operation.

Round Operation 2

Same as the Round Operation 1 as shown in section 3.1 but it has a different key K_2 as the Round Key operation for Round 2. The input for this round operation 2 is the previous round’s output. Round 2 also provides the same two 32-bit length outputs. This continues till the Round 16

Round Operation 16

This is the last round operation 16. Here the output of the Round Operation 16 LE_17 and RE_17 are swapped again which are 32-bits length.

Summary of Encryption and Round Operation:

Every Round Operation has a different Key K_1, K_2, and so on. The output LE_16 and RE_16 are the pre-output and once it is swapped it becomes Cipher text LE_17 and RE_17.

Decryption

This section explains the decryption process in the first structure. Decryption is a simple reverse operation of the encryption process where the Round 1 Operation of the decryption uses K_16, round 2 takes K_15 and finally, round 16 takes K_1. The LD_16 and RD_16 are not the final output or plain text. The LD_16 and RD_16 again swapped and the final output RD_17 and LD_17 is the plain text.

What is the ‘F’ Function?

We expect confusion and diffusion properties to make it robust. We must key or supply the key generated by the Key Scheduling Algorithm for each round operation.

In the function ‘F’ in every round operation, there are 2 important operations that are responsible for the confusion and the diffusion. **Confusion** obscures the relationship between the key and ciphertext, while **Diffusion** spreads the influence of plaintext bits across the ciphertext to hide patterns.

The F is responsible for carrying out the permutation and the substitution activities so that the cipher text we are getting will have confusion and diffusion properties.

STEGANOGRAPHY

A plaintext message may be hidden in any one of the two ways. The methods of steganography conceal the existence of the message, whereas the methods of cryptography render the message unintelligible to outsiders by various transformations of the text.

A simple form of steganography, but one that is time consuming to construct is one in which an arrangement of words or letters within an apparently innocuous text spells out the real message.

e.g., (i) the sequence of first letters of each word of the overall message spells out the real (Hidden) message.

(ii) Subset of the words of the overall message is used to convey the hidden message.

Various other techniques have been used historically, some of them are

Character marking – selected letters of printed or typewritten text are overwritten in pencil. The marks are ordinarily not visible unless the paper is held to an angle to bright light.

Invisible ink – a number of substances can be used for writing but leave no visible trace until heat or some chemical is applied to the paper.

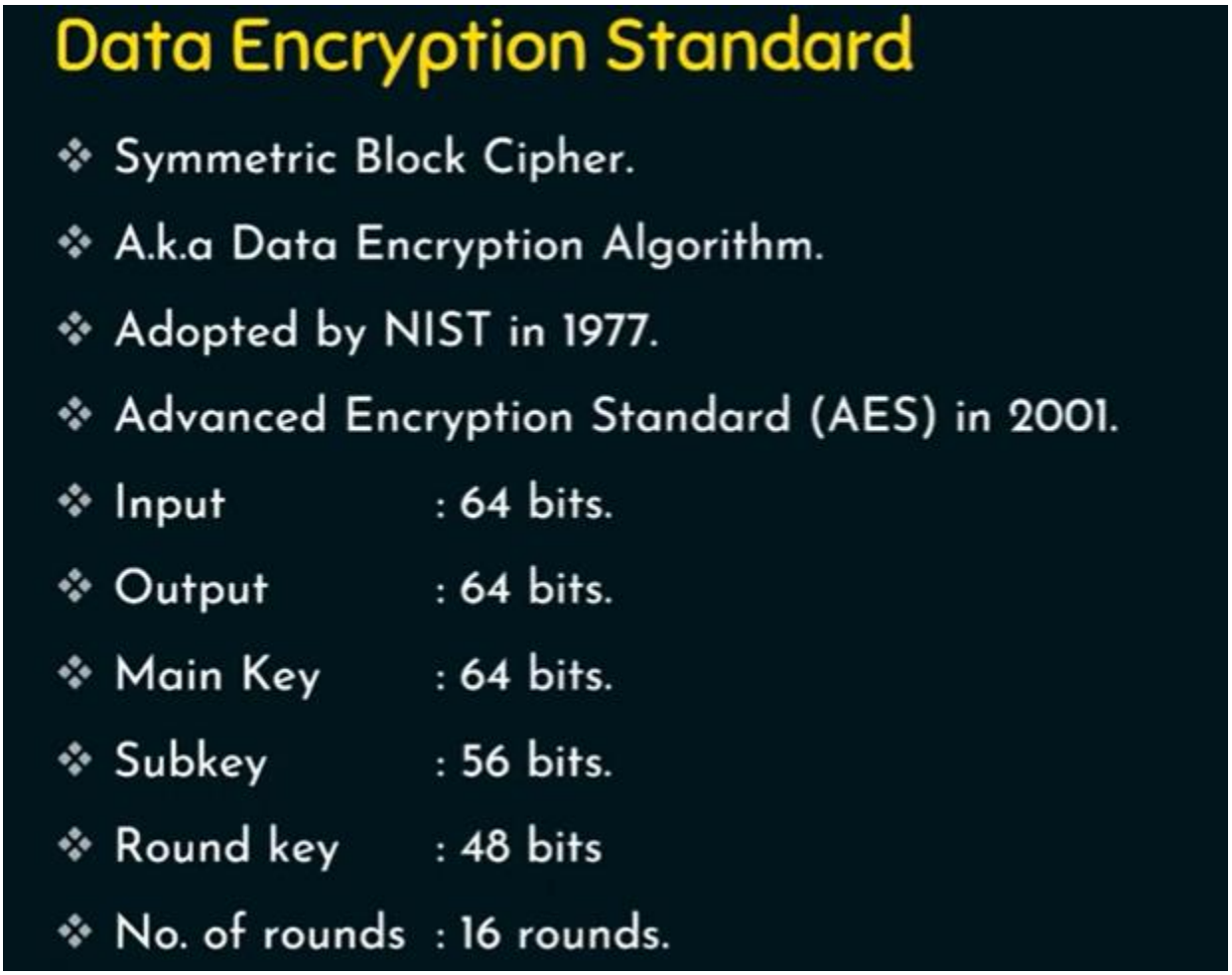
Pin punctures – small pin punctures on selected letters are ordinarily not visible unless the paper is held in front of the light. Typewritten correction ribbon – used between the lines typed

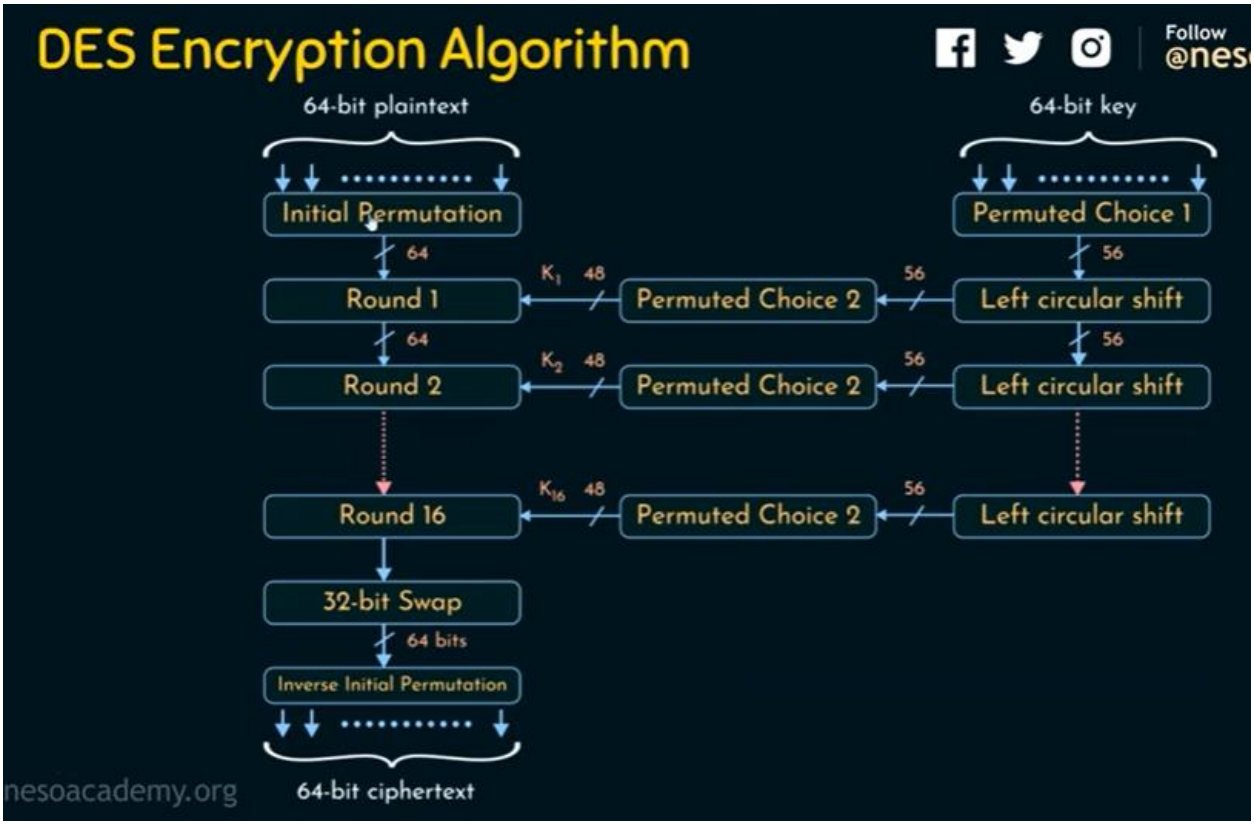
with a black ribbon, the results of typing with the correction tape are visible only under a strong light.

Drawbacks of steganography

Requires a lot of overhead to hide a relatively few bits of information.

Once the system is discovered, it becomes virtually worthless.





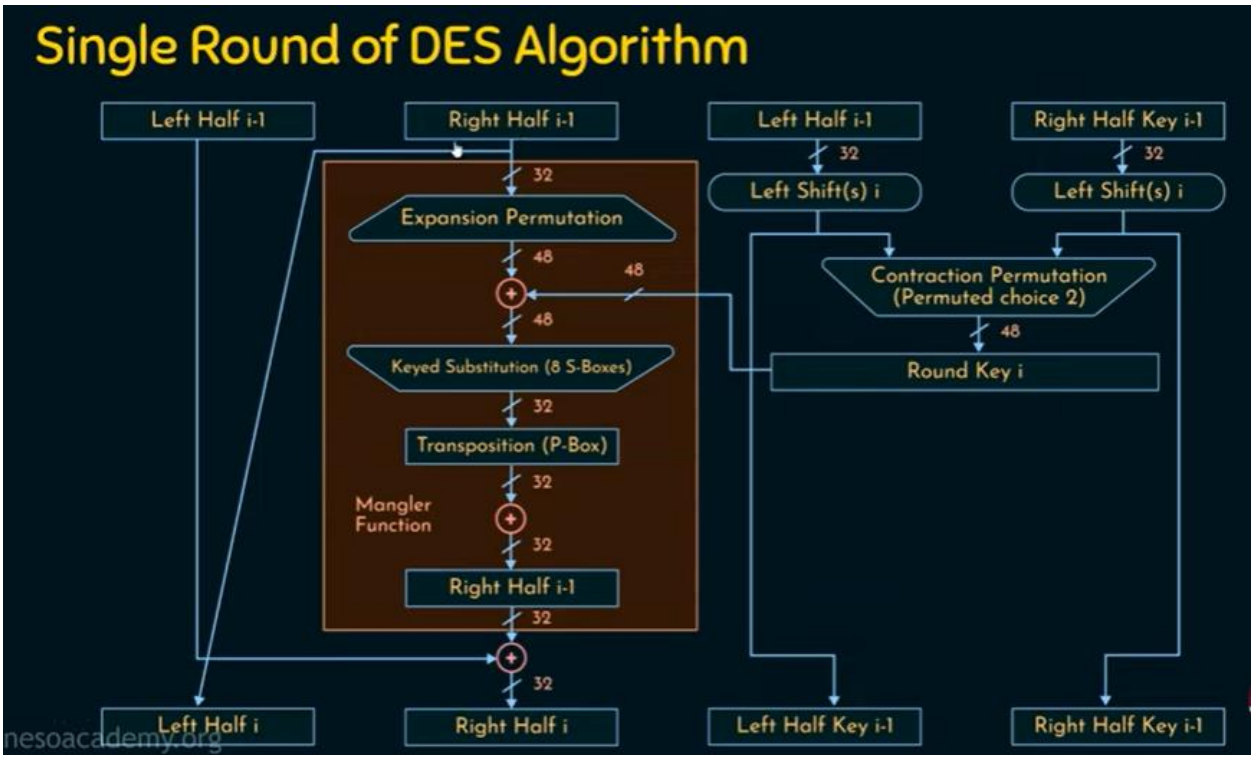
Initial Permutation

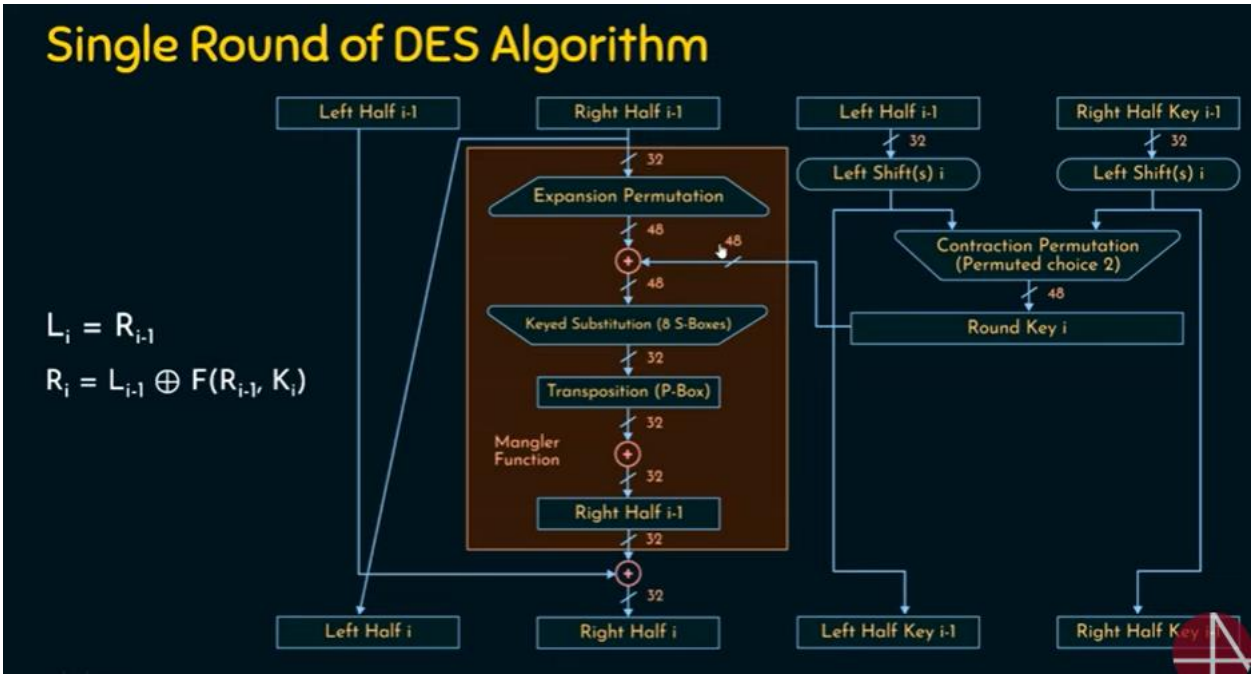
1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

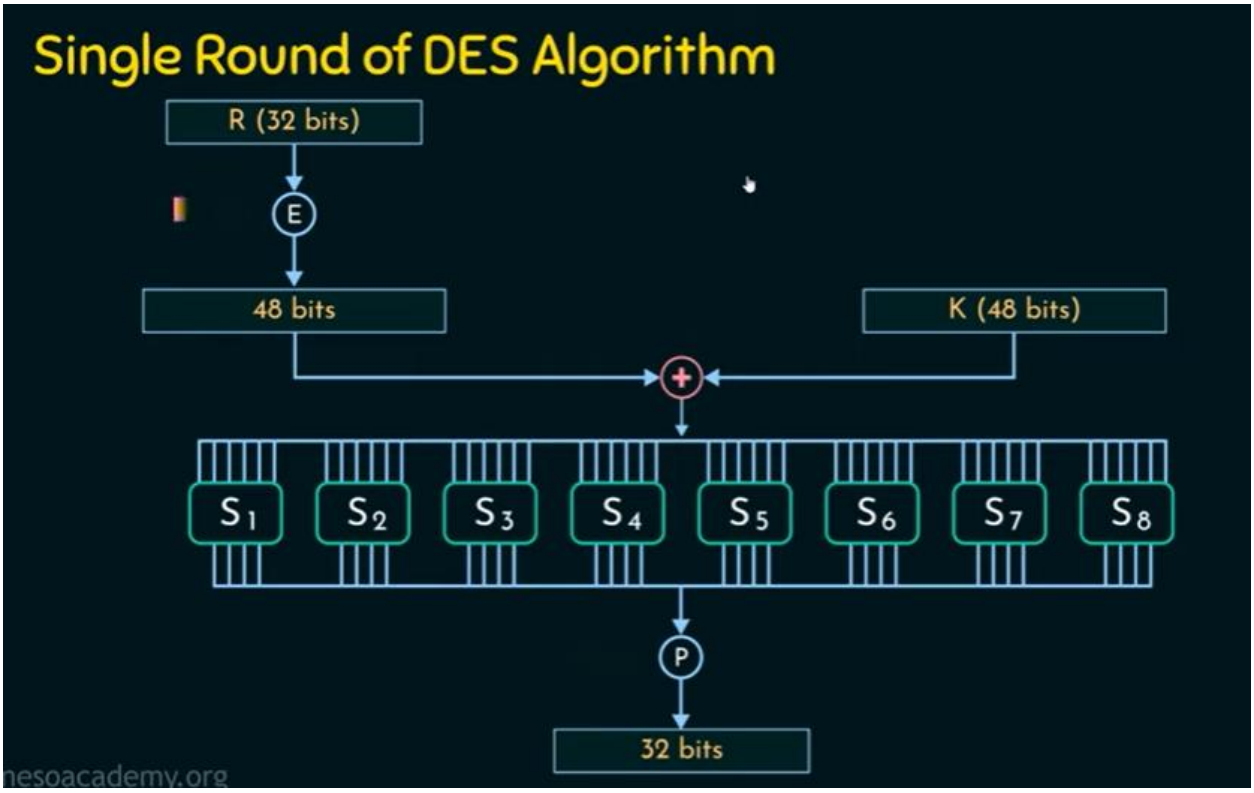
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Inverse Initial Permutation

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

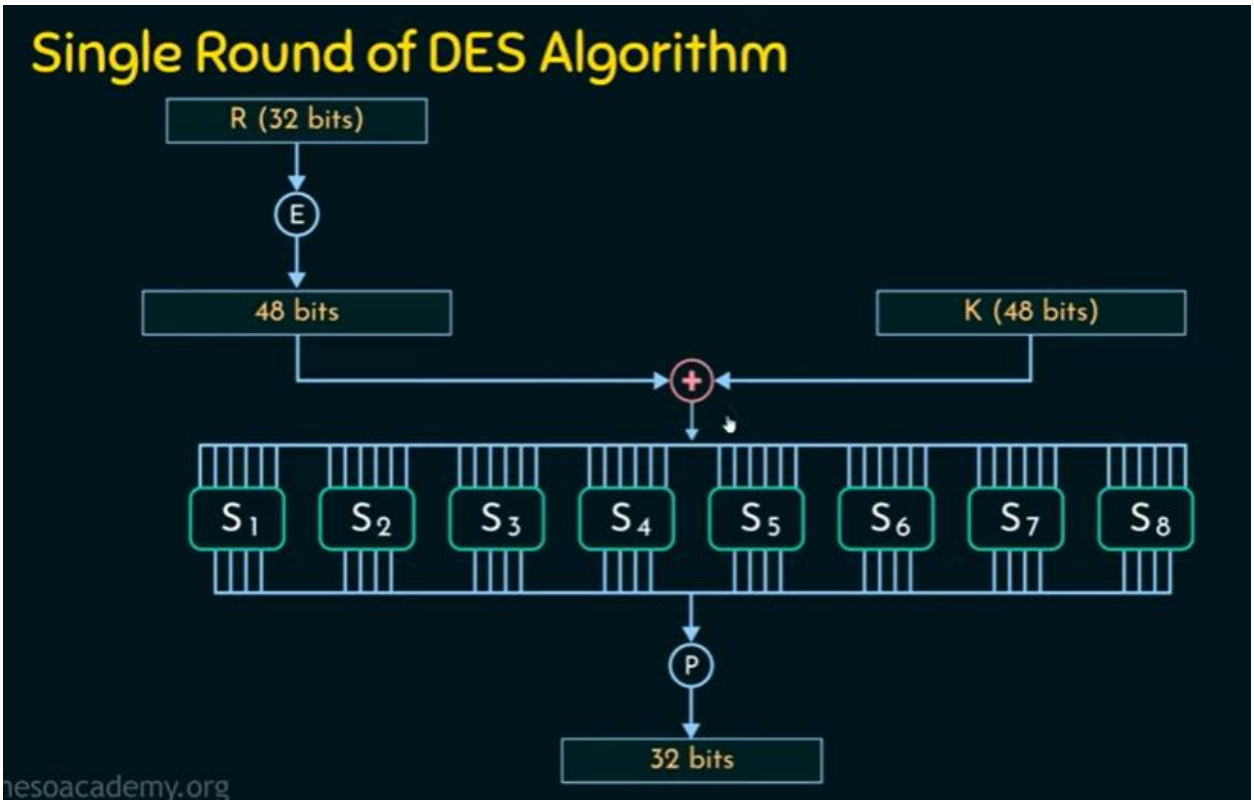

Single Round of DES Algorithm
Left Half i-1
Right Half i-1
Left Half i-1
Right Half Key i-1
32
32
Left Shift(s) i
Left Shift(s) i
Expansion Permutation
48
48
48
Contraction Permutation
(Permuted choice 2)
48
Keyed Substitution (8 S-Boxes)
Round Key i
32
Transposition (P-Box)
32
Mangler
Function
32
Right Half i-1
32
32
Left Half i
Right Half i
Left Half Key i-1
Right Half Key i-1
nesoacademy.org

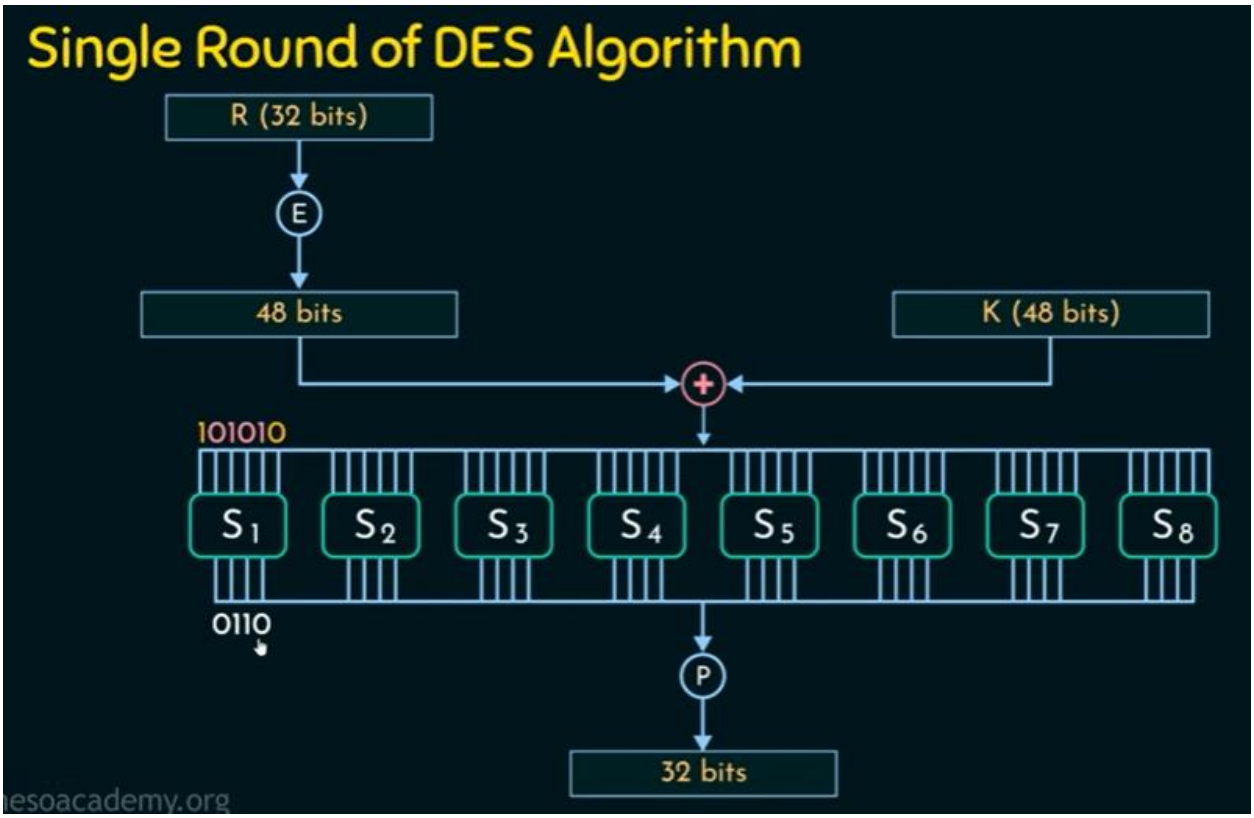

Single Round of DES Algorithm
$L_i = R_{i-1}$
$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$
Left Half i-1
Right Half i-1
Left Half i-1
Right Half Key i-1
32
32
Left Shift(s) i
Left Shift(s) i
Expansion Permutation
48
48
48
Contraction Permutation
(Permuted choice 2)
48
Keyed Substitution (8 S-Boxes)
Round Key i
32
Transposition (P-Box)
32
Mangler
Function
32
Right Half i-1
32
32
Left Half i
Right Half i
Left Half Key i-1
Right Half Key i-1


Single Round of DES Algorithm
R (32 bits)
E
48 bits
K (48 bits)
+
S1
S2
S3
S4
S5
S6
S7
S8
P
32 bits
nesoacademy.org

The Expansion Permutation

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1


Single Round of DES Algorithm
R (32 bits)
E
48 bits
K (48 bits)
+
S1
S2
S3
S4
S5
S6
S7
S8
P
32 bits
nesoacademy.org


Single Round of DES Algorithm
R (32 bits)
E
48 bits
K (48 bits)
+
101010
S1
S2
S3
S4
S5
S6
S7
S8
0110
P
32 bits
nesoacademy.org

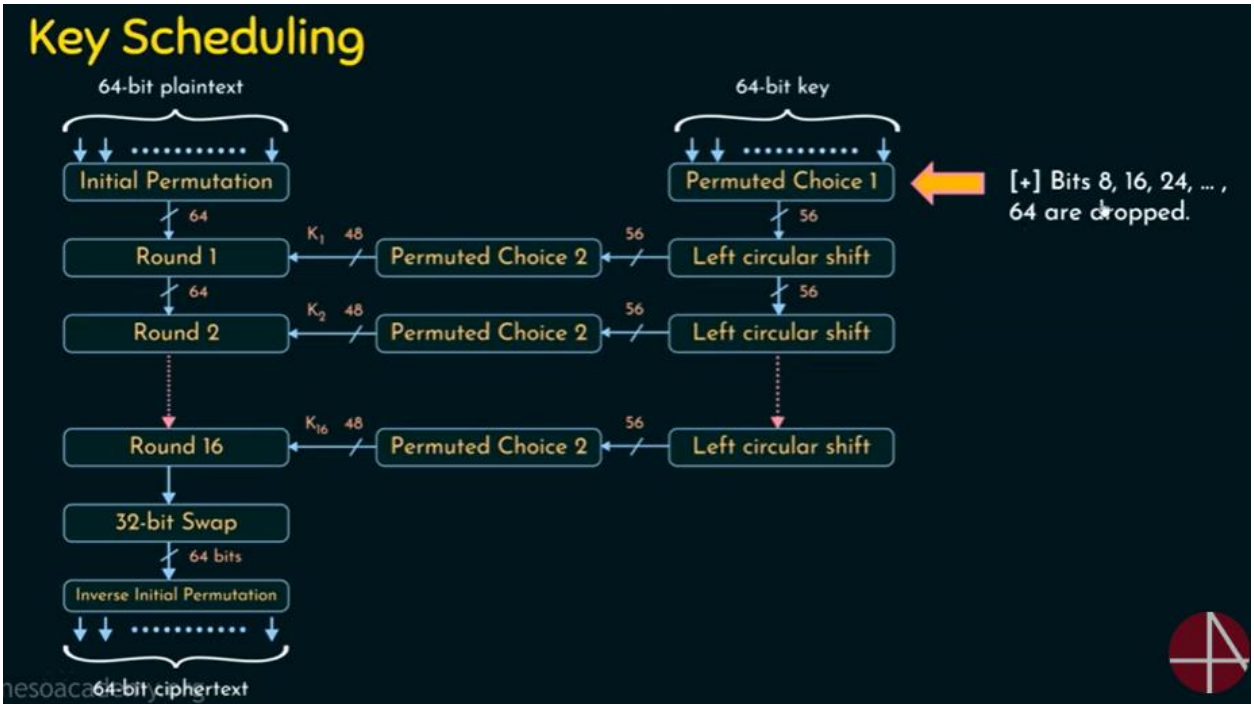
Box S_1

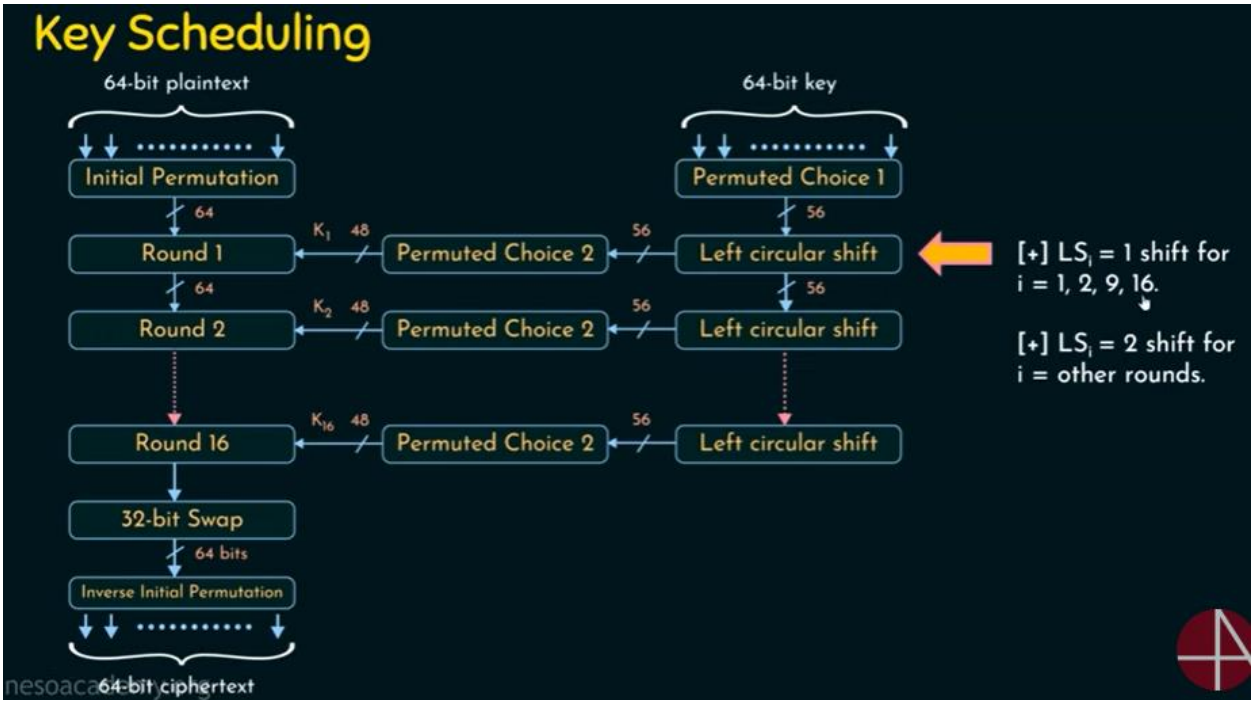
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
00	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
01	0	15	7	4	14	2	13	1	10	6	12	11	6	5	3	8
10	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
11	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

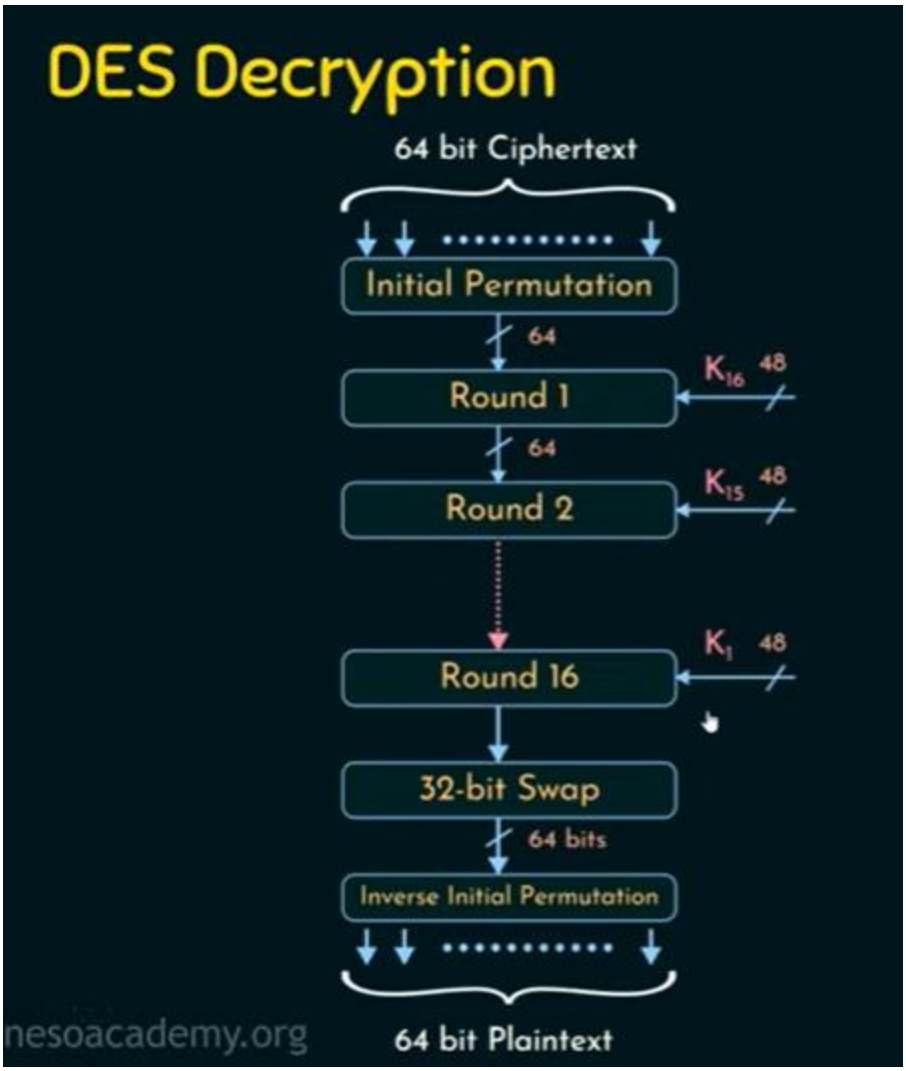
For example, $S_1(101010) = 6 = 0110$.

The Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25


Key Scheduling
64-bit plaintext
64-bit key
Initial Permutation
Permuted Choice 1
[+] Bits 8, 16, 24, ... , 64 are dropped.
64
56
Round 1
K1 48
Permuted Choice 2
56
Left circular shift
Round 2
K2 48
Round 16
K16 48
32-bit Swap
64 bits
Inverse Initial Permutation
64-bit ciphertext


Key Scheduling
64-bit plaintext
64-bit key
Initial Permutation
Permuted Choice 1
64
56
Round 1
K1 48
Permuted Choice 2
56
Left circular shift
[+] LSi = 1 shift for i = 1, 2, 9, 16.
[+] LSi = 2 shift for i = other rounds.
Round 2
K2 48
Round 16
K16 48
32-bit Swap
64 bits
Inverse Initial Permutation
64-bit ciphertext


DES Decryption
64 bit Ciphertext
Initial Permutation
64
Round 1
K16 48
64
Round 2
K15 48
Round 16
K1 48
32-bit Swap
64 bits
Inverse Initial Permutation
64 bit Plaintext
nesoacademy.org

Implementing DES Algorithm (For Encryption)

To understand the DES algorithm, let's use a simple example. We'll start with a 64-bit block of plaintext and a 56-bit key.

We'll walk through each step, explaining the process in detail.

Initial Permutation (IP)

The first step in DES is the Initial Permutation (IP). This step rearranges the bits of the plaintext based on a predefined table. Let's say our plaintext is:

- **Plaintext (in Hexadecimal):** 0123456789ABCDEF

We convert the hex to binary:

- **Plaintext (in binary):** 00000001 00100011 01000101 01100111 10001001 10101111 11001101 11101111

The Initial Permutation table, which is pre-defined, is:

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3

61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

To perform the initial permutation, we rearrange the bits of the plaintext according to the table. For instance, the bit in position 58 of the original plaintext moves to position 1 of the permuted text.

Permutation example:

- Bit at position 58 (original) -> Bit at position 1 (permuted)
- Bit at position 50 (original) -> Bit at position 2 (permuted)

…continue for all 64 bits.

- **After Initial Permutation (example):** 11001100 11110000 10101010 11001100 10101010 11001100 11110000 11001100

Splitting

Next, we split the permuted plaintext into two halves. Each half is 32 bits.

- **Left half (L0):** 11001100 11110000 10101010 11001100
- **Right half (R0):** 10101010 11001100 11110000 11001100

Key Generation

The initial 64-bit key, which you can choose randomly, is transformed into a 56-bit key by discarding every 8th bit. This process reduces the key to 56 bits, which is then divided into two 28-bit halves.

- **Initial Key (in hex):** 133457799BBCDFF1

Convert to binary and drop every 8th bit:

- **Key (in binary):** 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

Drop every 8th bit:

- **56-bit Key:** 00010010 01101001 01011011 11001001 10110111 10110111 11110001

In DES, each of the 16 rounds uses a different 48-bit sub-key derived from the original 56-bit key. This process adds complexity and ensures that the encryption is secure.

48-bit key generation process for each round

Splitting into Halves:

The 56-bit key is divided into two 28-bit halves:

- **C0 (first 28 bits):** 00010010 01101001 01011011 1100 (28 bits)
- **D0 (second 28 bits):** 1001 10110111 10110111 11110001 (28 bits)

Shifting:

For each of the 16 rounds, the halves are circularly shifted left by one or two positions, depending on the round number. The number of shifts per round is predetermined as follows:

Round	Shifts
1	1
2	1
3	2
4	2
5	2
6	2
7	2
8	2
9	1
10	2
11	2
12	2
13	2
14	2
15	2
16	1

Example for Rounds 1 and 2:

Round 1:

- Shift C0 and D0 left by 1 position.
- C1: 00100100 11010010 10110111 1000
- D1: 00110111 01111001 10111111 1111

Round 2:

- Shift C1 and D1 left by 1 position.
- C2: 01001001 10100101 01101111 0000
- D2: 01101110 11110011 01111111 1110

This shifting process is repeated for all 16 rounds.

Compression Permutation

After shifting, the 56-bit key (combined C and D halves) is compressed to 48 bits using a permutation table. This table selects 48 specific bits out of the 56-bit combined key to create the 48-bit round key.

Compression Permutation Table:

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

This table is used to select and reorder the bits from the combined 56-bit key to produce the 48-bit round key.

Example:

Assume the combined 56-bit key after shifting for round 1 is:

- **Combined C1 and D1:** 00100100 11010010 10110111 10000011 01111001 10111111

Apply the compression permutation to select and rearrange the bits:

- **48-bit Round Key (example bits selected):** 111000110010001010110010000100110011010000101111

By repeating this process for each round, we generate a unique 48-bit round key for each of the 16 rounds, which are then used in the encryption process.

Rounds (Round 1 as an Example)

DES involves **16 rounds** of key transformations. For each round, a 48-bit sub-key is generated from the 56-bit key.

After splitting the blocks, we use the round function. In each round, the left half remains as it is, and we need to perform the operations below on the right half.

1. Expansion Permutation (E)
2. Key mixing ($\oplus$
3. Substitution (S1, S2,…,S8)
4. Permutation (P)

After getting the final result from the right half, we need to perform an XOR operation of the left and right half.

Expansion Permutation (E)

The 32-bit right half (R0) is expanded to 48 bits using the expansion table. Here, each bit is substituted with the one given at the respected position in the table. The expansion table duplicates certain bits to increase the size from 32 to 48 bits.

Expansion Table:

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Apply the expansion permutation on R0:

- **R0 (32 bits):** 10101010 10101010 10101010 10101010
- **Expanded R0 (48 bits):** 01010101 01011010 10100101 01101010 01011010 10101010

Key Mixing (⊕)

The expanded R0 is XORed with the round key (48-bit). Using the example from the previous step, our round key for Round 1 is:

- **Round Key (48 bits):** 111000110010001010110010000100110011010000101111

Perform XOR operation:

- **Result:** 10110101 00010000 00011111 01111000 01010000 00011101

Substitution (S-Boxes):

The result is divided into eight 6-bit blocks, and each block is substituted using the S-Boxes. Here's an example using S1:

S1 Table:

Each 6-bit block is used to look up the corresponding 4-bit output from the S-Box. For example, you need to take a look at the first sub-table. Let's take the example of the first 6-bit block:

First 6-bit block: 101101

- **Row:** 11 (first and last bits)
- **Column:** 0110 (middle four bits)
- **Output (S1[11][0110]):** 0100

Perform this for all 8 blocks and combine the results to get a 32-bit block.

Permutation (P)

The 32-bit result of the S-Boxes is permuted using a fixed table:

Permutation Table:

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Applying this permutation provides the final 32-bit output of the round function (F).

XOR Operation

After getting the final result from the right half, we need to perform an XOR operation of the left half (L0) and the result from the right half.

- **Final 32-bit output from F:** <32-bit result from P>
- **L0 (32 bits):** 11001100 00000000 11001100 11110000
- **XOR Result:** <L0 XOR 32-bit result from F>

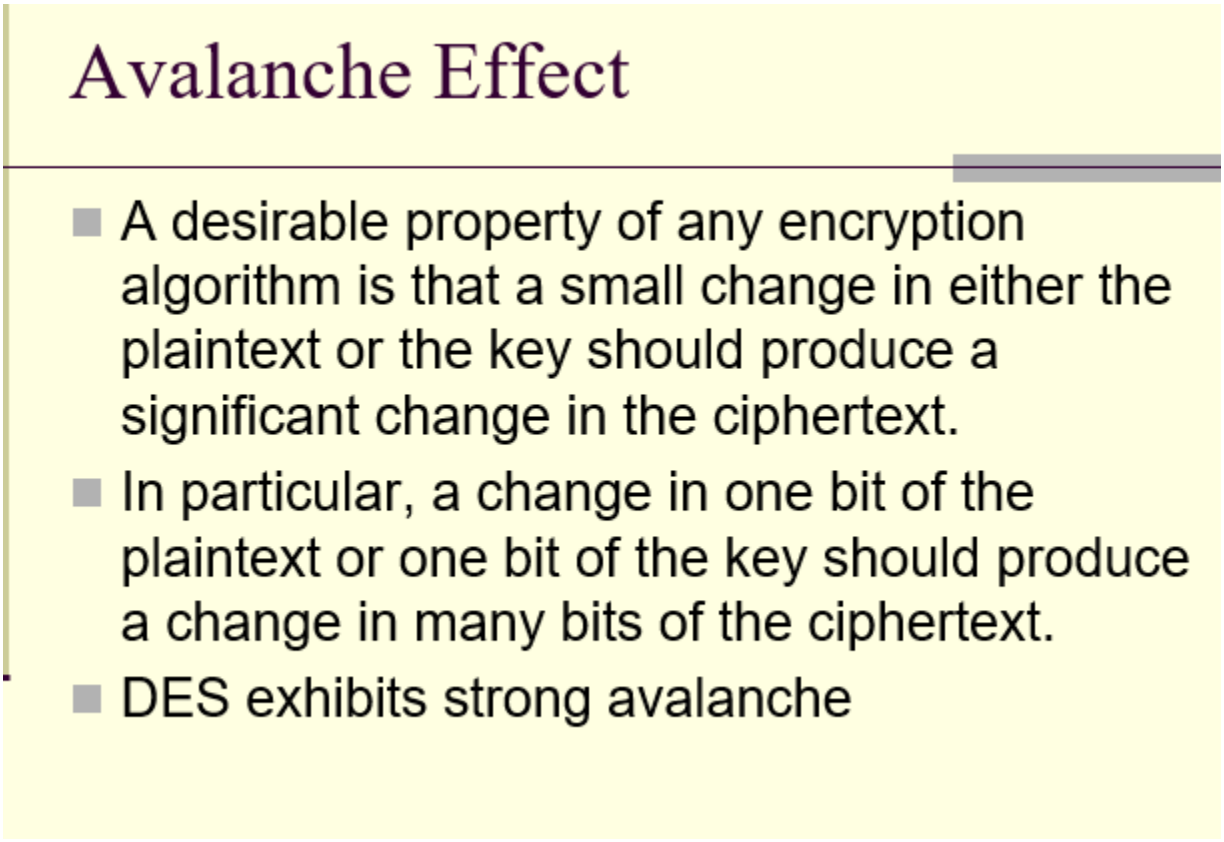
Result after Round 1 (Swapping required)

The result after Round 1 is the combination of the original right half (R0) and the XOR result:

- **L1**: <Original R0>
- **R1:** <XOR Result>

This process is repeated for all 16 rounds. Each round uses a different 48-bit sub-key, and the final result is obtained after the 16th round, followed by the final permutation (FP).

By following these steps, the plaintext is transformed into ciphertext using DES.



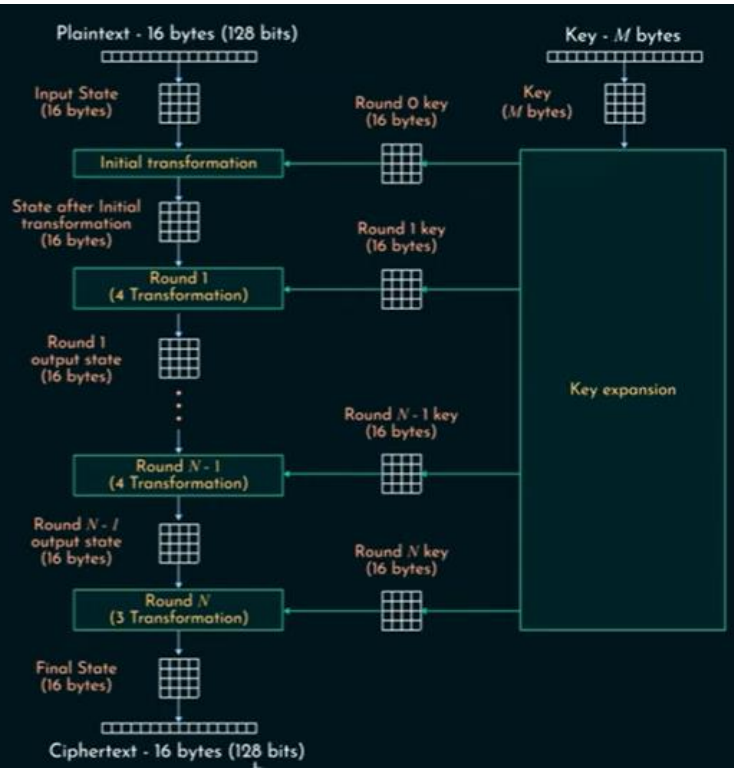
Avalanche Effect in DES

- A desirable property of any encryption algorithm.
- What is avalanche effect?
- DES – Strong Avalanche Effect.
  - 1 bit change in PT – 34 bits change in CT on average.
  - 1 bit change in Key – 35 bits changes in CT on average.

AES

- Advanced Encryption Standard.
- NIST in 2001.
- Symmetric block cipher.
- Widely used.

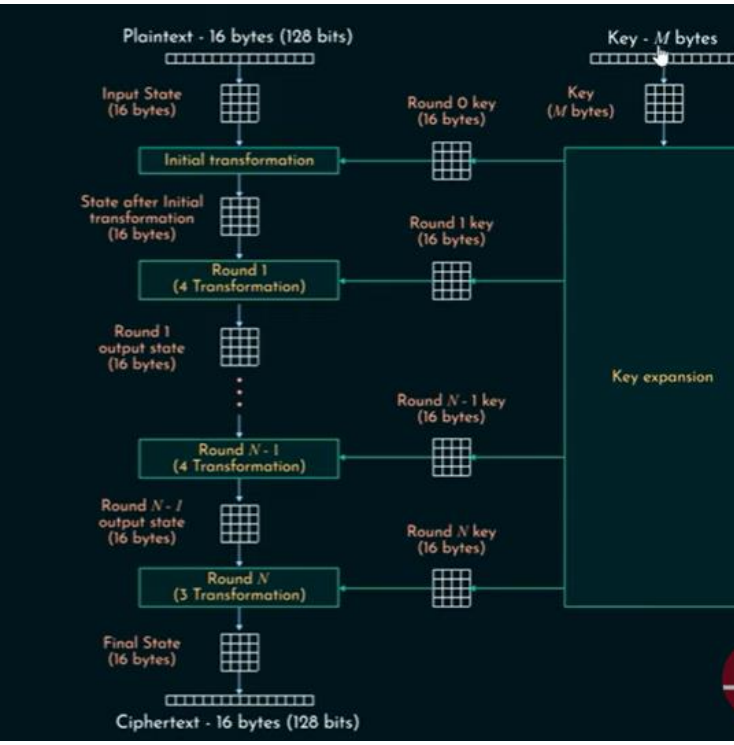
AES Structure



nesoacademy.org

AES Structure

No. of rounds	Key size (in bits)
10	128
12	192
14	256



nesoacademy.org

AES Parameters

	AES-128	AES-192	AES-256
Key Size	128	192	256
Plaintext Size	128	128	128
Number of rounds	10	12	14
Round Key Size	128	128	128